

```
julia> flights = loadtable("hflights.csv")
Table with 227496 rows, 21 columns:
Columns:
#   colname          type
-----
1   Year             Int64
2   Month             Int64
3   DayOfMonth        Int64
4   DayOfWeek         Int64
5   DepTime           Union{Missing, Int64}
6   ArrTime           Union{Missing, Int64}
7   UniqueCarrier     String
8   FlightNum         Int64
9   TailNum           String
10  ActualElapsedTime Union{Missing, Int64}
11  AirTime            Union{Missing, Int64}
12  ArrDelay           Union{Missing, Int64}
13  DepDelay           Union{Missing, Int64}
14  Origin             String
15  Dest               String
16  Distance           Int64
17  TaxiIn             Union{Missing, Int64}
18  TaxiOut            Union{Missing, Int64}
19  Cancelled          Int64
20  CancellationCode   String
21  Diverted           Int64
```

Figure 3: Loadtable screenshot

```
(i.DayOfMonth == 22), flights)
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
```

Obviously, this function matches rows where the month is 10 and DayOfMonth is 22. You can combine conditions as shown below:

```
filter(i -> (i.UniqueCarrier == "AA") ||
(i.UniqueCarrier == "UA"), flights)
1231     "HOU"  "SAT"
124      "HOU"  "STL"
280      "HOU"  "STL"
782      "HOU"  "STL"
1050     "HOU"  "STL"
201      "HOU"  "TPA"
471      "HOU"  "TPA"
1191     "HOU"  "TPA"
1674     "HOU"  "TPA"
127      "HOU"  "TUL"
621      "HOU"  "TUL"
1597     "HOU"  "TUL"
```

Selecting specific columns

Specific columns can be selected with column names. A sample *Select* function and its output are shown below:

```
julia> select(flights, (:FlightNum,
:Origin, :Dest))
Table with 227496 rows, 3 columns:
FlightNum Origin Dest
```

```
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
428      "IAH"  "DFW"
```

Nesting and piping

When you want to carry out multiple operations one after another, it can be done in two ways — by nesting and/or piping.

In the following code snippet, we are nesting by filtering for departure delays of more than 40 (*!ismissing*

is used to detect missing values and select two columns).

```
filter(i -> !ismissing(i.DepDelay >
40), select(flights, (:UniqueCarrier,
:DepDelay)))
```

Piping can be achieved with the *Lazy* package.

```
import Lazy
Lazy.@as x flights begin
    select(x, (:UniqueCarrier,
:DepDelay))
    filter(i -> !ismissing(i.DepDelay >
60), x)
end
```

Sorting

Sorting can be carried out with the *Sort* function, as shown below:

```
julia> sort(flights, :FlightNum, select
= (:FlightNum, :Origin, :Dest))
Table with 227496 rows, 3 columns:
FlightNum Origin Dest
```

```
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
1      "IAH"  "HNL"
7037   "IAH"  "IAD"
7037   "IAH"  "IAD"
7037   "IAH"  "IAD"
7037   "IAH"  "IAD"
7037   "IAH"  "IAD"
7231   "IAH"  "IAD"
7240   "IAH"  "IAD"
7240   "IAH"  "IAD"
7240   "IAH"  "IAD"
7240   "IAH"  "IAD"
7240   "IAH"  "IAD"
```